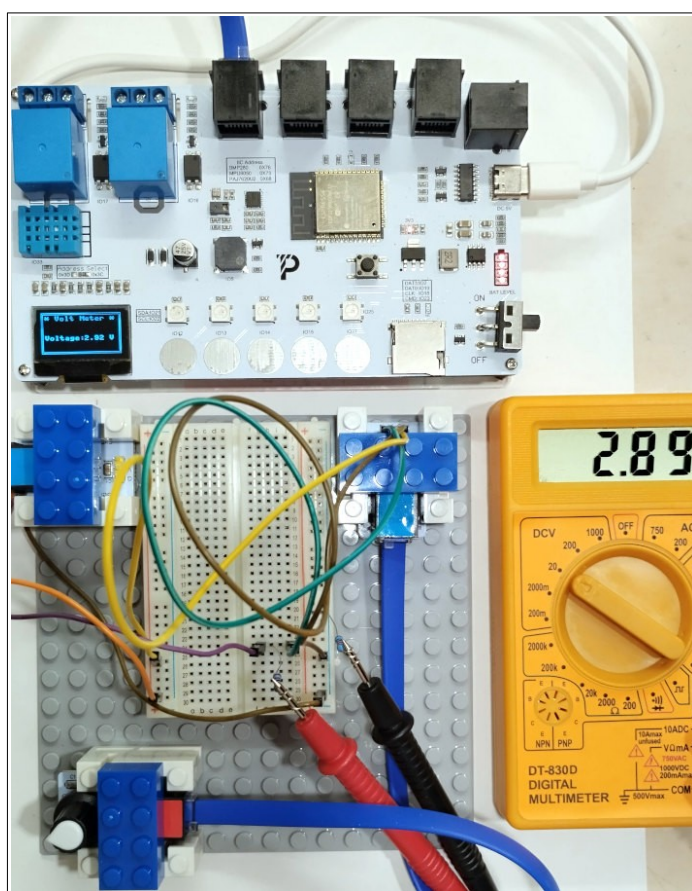


Άσκήσεις Micropython

στο σύστημα ARD:Icon II της Polytech



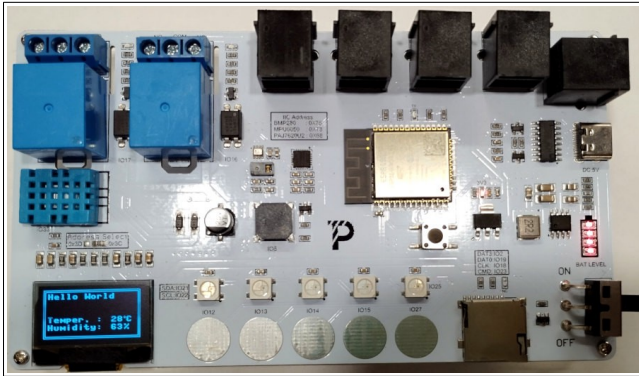
Έκδοση 1.0

Σταύρος Σ. Φώτογλου ΠΕ86
Ε.Κ. Πρέβεζας – 1ο ΕΠΑ.Λ. Πρέβεζας

Πρέβεζα 2025

Άσκηση 1

Στην πλακέτα ARD:icon II (IOT) του kit ρομποτικής S4T της Polytech, να γίνει πρόγραμμα σε Micropython το οποίο εμφανίζει την θερμοκρασία και την σχετική υγρασία από τον αισθητήρα DHT 11 στην ενσωματωμένη οθόνη OLED του συστήματος. Οι τιμές θα ανανεώνονται κάθε 2 δευτερόλεπτα.



Μέσα στο σύστημα αρχείων πρέπει να αντιγραφτεί η βιβλιοθήκη `ssd1306.py`

```
1 #Πρόγραμμα το οποίο παρουσιάζει μετρήσεις Θερμοκρασίας και υγρασίας από DHT11 στην οθόνη OLED
2 import machine, time, dht
3 i2c = machine.I2C(0, sda = machine.Pin(21), scl = machine.Pin(22), freq = 400000)
4 #Η εναλλακτικά
5 #i2c = machine.SoftI2C(sda=machine.Pin(21), scl=machine.Pin(22))
6 from ssd1306 import SSD1306_I2C
7 sensor = dht.DHT11(machine.Pin(33))
8 sensor.measure()
9
10 #display = SSD1306_I2C(128, 64, i2c) #Δίνω μπορώ να βάλω διεύθυνση π.χ. SSD1306_I2C(128, 64, i2c, 0x3c)
11 display = SSD1306_I2C( width=128, height=64, i2c=i2c, addr=0x3c, external_vcc=False )
12 display.fill(0) #Καθαρίζει οθόνη
13 #Δημιουργώ ετικέτες
14 display.text('Hello World', 5, 5, 1) #Θέση x, y, color 1/0
15 display.text("Temper. : ", 5, 35, 1)
16 display.ellipse(106, 36, 1, 1, 1)
17 display.text("C", 108, 35, 1)
18 display.text("Humidity: ", 5, 45, 1)
19 display.text("%", 105, 45, 1)
20 #Δημιουργώ διπλό περίγραμμα
21 display.rect(0,0,127,63,1,0) #x, y, width, height, color=0/1, fill=0/1
22 display.rect(2,2,123,59,1,0) #x, y, width, height, color=0/1, fill=0/1
23 display.show() #Παρουσιάζει στην οθόνη
24
25 #Για πάντα
26 while True:
27     sensor.measure()
28     display.rect(80, 34, 25, 9, 0, True) #Σβήσε παλιά τιμή
29     display.text("{: 3}".format(sensor.temperature()), 80, 35, 1) #Εμφάνισε νέα τιμή
30     display.rect(80, 44, 25, 9, 0, True) #Σβήσε παλιά τιμή
31     display.text("{: 3}".format(sensor.humidity()), 80, 45, 1) #Εμφάνισε νέα τιμή
32     print("Θερμοκρασία:", sensor.temperature()) #Εμφάνισε στο τερματικό
33     print("Σχετ. Υγρασία:", sensor.humidity())
34     display.show() #Παρουσιάζει στην οθόνη
35     time.sleep(2) #Περίμενε 2 sec
```

Άσκηση 2

Να τροποποιηθεί το προηγούμενο ώστε να εμφανίζει και μετρήσεις του αισθητήρα BMP280. Ο αισθητήρας BMP280 μπορεί να μετρήσει ατμοσφαιρική πίεση και θερμοκρασία.

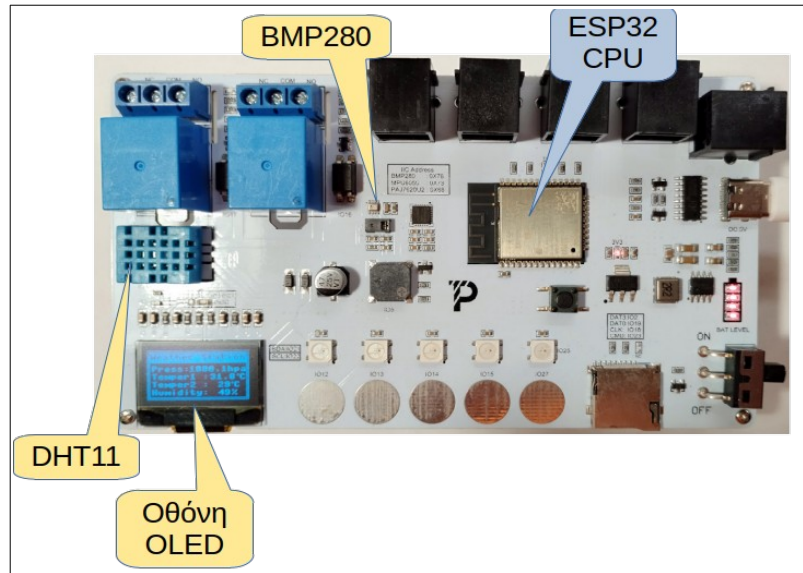
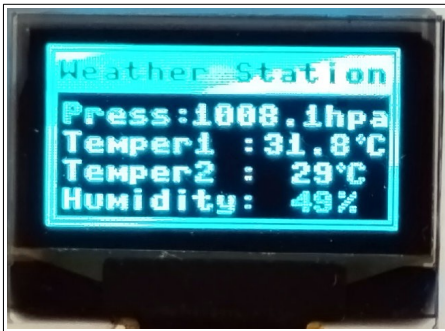
Μέσα στο σύστημα αρχείων πρέπει να αντιγραφτεί η βιβλιοθήκη `ssd1306.py` και η βιβλιοθήκη `bmp280.py`.

```
1 # Πρόγραμμα το οποίο παρουσιάζει μετρήσεις Θερμοκρασίας και υγρασίας από DHT11
2 # και ατμ. πίεσης και θερμοκρασίας από το BMP280 στην οθόνη OLED
3 import machine, time, dht, math
4 i2c = machine.I2C(0, sda = machine.Pin(21), scl = machine.Pin(22), freq = 400000)
5 #Η εναλλακτικά
6 #i2c = machine.SoftI2C(sda=machine.Pin(21), scl=machine.Pin(22))
7 from ssd1306 import SSD1306_I2C #0 ελεγκτής της οθόνης OLED
8 sensor = dht.DHT11(machine.Pin(33)) #0 αισθητήρας DHT11
9 from bmp280 import *
10 bmp = BMP280(i2c) #Στιγμιότυπο του BMP280
11 #Στιγμιότυπο της οθόνης
12 display = SSD1306_I2C(width=128, height=64, i2c=i2c, addr=0x3c, external_vcc=False)
13 display.fill(0) #Καθαρίζει οθόνη
14
15 #Δημιουργώ διπλό περίγραμμα
16 display.rect(0, 0, 127, 63, 1, 0) #x, y, width, height, color=0/1, fill=0/1
17 display.rect(2, 2, 123, 59, 1, 0) #x, y, width, height, color=0/1, fill=0/1
18
```

```

19 #Δημιουργίες ετικέτες
20 display.rect(3, 2, 121, 14, 1, 1) #x, y, width, height, color=0/1, fill=0/1
21 display.text('Weather Station', 3, 5, 0) #θέση x, y, color 1/0
22 display.text("Press: ", 5, 20, 1)
23 display.text("hpa", 100, 20, 1)
24 display.text("Temper1 : ", 5, 30, 1)
25 display.ellipse(113, 31, 1, 1, 1)
26 display.text("C", 115, 30, 1)
27 display.text("Temper2 : ", 5, 40, 1)
28 display.ellipse(106, 41, 1, 1, 1)
29 display.text("C", 108, 40, 1)
30 display.text("Humidity: ", 5, 50, 1)
31 display.text("%", 105, 50, 1)
32 display.show() #Παρουσίασε στην οθόνη
33
34 #Για πάντα
35 while True:
36     bmp.force_measure() #Διάβασε τιμές από BMP280
37     display.rect(52, 19, 48, 9, 0, True) #Εβήσε παλιά τιμή ατμ. πίεσης
38     display.text(str(round(bmp.pressure / 100), 1)), 52, 20, 1) #Εμφάνισε νέα τιμή ατμ. πίεσης
39     display.rect(77, 29, 34, 9, 0, True) #Εβήσε παλιά τιμή θερμοκρασίας BMP
40     display.text(str(round(bmp.temperature, 1)), 78, 30, 1) #Εμφάνισε νέα τιμή θερμοκρασίας BMP
41     print("Θερμοκρασία BMP: " + str(round(bmp.temperature, 1))) #Εμφάνισε σε τερματικό
42     print("Πίεση: " + str(round(bmp.pressure / 100), 1)) #Εμφάνισε σε τερματικό
43
44     sensor.measure() #Διάβασε τιμές από DHT11
45     display.rect(80, 39, 25, 9, 0, True) #Εβήσε παλιά τιμή Θερμοκρασίας DHT
46     display.text("{: 3}".format(sensor.temperature()), 80, 40, 1) #Εμφάνισε νέα τιμή θερμοκρ. DHT
47     display.rect(80, 49, 25, 9, 0, True) #Εβήσε παλιά τιμή σχετ. υγρασίας
48     display.text("{: 3}".format(sensor.humidity()), 80, 50, 1) #Εμφάνισε νέα τιμή υγρασίας
49     print("Θερμοκρασία DHT:", sensor.temperature()) #Εμφάνισε στο τερματικό
50     print("Σχετ. Υγρασία:", sensor.humidity()) #Εμφάνισε στο τερματικό
51     print() #Άσε μια γραμμή κενή
52
53     display.show() #Παρουσίασε στην οθόνη
54     time.sleep(2) #Περίμενε 2 sec

```



Ασκηση 3

Να γίνει πρόγραμμα σε Micropython το οποίο θα ανιχνεύει το άγγιγμα των κουμπιών IO12 και IO13. Όταν πατιέται το IO12 θα ανάβει το RGB Led πάνω απ' αυτό σε λευκό χρώμα και όταν πατιέται το IO13 θα σβήνει. Κατά το πάτημα των κουμπιών θα ακούγεται και χαρακτηριστικός ήχος από τον βομβητή.

Για την υλοποίηση δεν χρειάζονται επιπλέον βιβλιοθήκες.

```

1 import machine, time, neopixel
2
3 #Ορίσμοι
4 threshold = 200 # Threshold to be adjusted
5 Debounce = 50 #msec
6
7 #Στιγμιότυπο RGB Leds
8 rgb = neopixel.NeoPixel(machine.Pin(25), 5) #IO25, 5 x RGB Leds
9
10 #Ορίσμος ακροδεκτών κουμπιών αφής
11 t_pin1 = machine.TouchPad(machine.Pin(12)) #1ο κουμπί αφής
12 t_pin2 = machine.TouchPad(machine.Pin(13)) #2ο κουμπί αφής

```

```

13
14 #Ορισμός ακροδέκτη Beeper
15 beeper = machine.Pin(5, machine.Pin.OUT) #Ο βομβητής συνδέεται στο pin IO5
16 #Συνάρτηση παραγωγής τόνου συχνότητας freq σε Hz και διάρκειας dur σε msec
17 def beep(freq = 500, dur = 500): #Προκαθορισμένο 500Hz, 500msec
18     #Υπολογισμοί
19     period = 1.0 / freq #Περίοδος
20     half_per = int((period / 2) * 1000000) #Ημιπερίοδος σε μsecs
21     times = int(dur / 1000 / period) #Αριθμός κύκλων
22     for i in range(times):
23         beeper.value(1)
24         time.sleep_us(half_per)
25         beeper.value(0)
26         time.sleep_us(half_per)
27
28 #----- Ορισμός κλάσης ελέγχου κουμπιών αφής (Μόνο απλό κλικ) -----
29 class Touch:
30     #Κατασκευαστής
31     def __init__(self, pin):
32         self.pin = pin #Το GPIO του κουμπιού αφής
33         self.validclick = False #Έγκυρο πάτημα
34         self.timer = 0 #Ο timer που κρατάει τα ticks σε msec
35         self.downtime = 0 #Πόσο χρόνο είναι πατημένο (Δάχτυλο πάνω στο κουμπί αφής)
36
37     #Συμβάν απλό κλικ
38     def onClick(self):
39         self()
40
41     #Καλείται περιοδικά και ελέγχει το κάθε κουμπί
42     def checkButton(self):
43         self.timer = time.ticks_ms() #Κράτησε τον χρόνο συστήματος
44         cVal = self.pin.read() #Διάβασε τιμή του αισθητήρα touch
45         state = False #Δεν πατήθηκε
46         if cVal < threshold: #Αν είναι κάτω από το κατώφλι (Δάχτυλο πάνω στην νησίδα)
47             state = True #Πατήθηκε το κουμπί
48         #----- Πατημένο -----
49         #Πατήθηκε τώρα για πρώτη φορά και ο χρόνος δεν μετράει
50         if state and self.downtime == 0:
51             self.downtime = self.timer #Αρχισε να μετράς τον χρόνο που είναι πατημένο - downtime
52         #Παραμένει πατημένο και έλεγξε αν πέρασε ο χρόνος debounce και δεν έχει ενεργοποιηθεί valid click
53         elif state and not self.validclick and (self.timer - self.downtime) > Debounce:
54             self.validclick = True #Να μην ξαναμπείς εδώ
55             self.onClick() #Κάλεσε συνάρτηση εξυπηρέτησης
56         #----- Ελεύθερο -----
57         #Αλλιώς αν ήταν πατημένο και τώρα το άφησε
58         elif not state:
59             self.downtime = 0 #Επαναφορά
60             self.validclick = False #Επαναφορά για την επόμενη φορά
61         #----- Τέλος ορισμού κλάσης -----
62
63 #----- Συναρτήσεις εξυπηρέτησης συμβάντων -----
64 def key1_click(): #Αναψε
65     print("Led is ON")
66     #----- R, G, B -----
67     rgb[4] = (255, 255, 255) #Τιμές από 0 (σβηστό) έως 255 (μέγιστη ένταση)
68     rgb.write()
69     beep(2000, 50)
70
71 def key2_click(): #Σβήσε
72     print("Led is OFF")
73     rgb[4] = (0, 0, 0)
74     rgb.write()
75     beep(400, 50)
76
77 #===== Κυρίως πρόγραμμα =====
78 #----- Δημιουργία στιγμιοτύπων και ορισμός συναρτήσεων εξυπηρέτησης -----
79 ts1 = Touch(t_pin1)
80 ts1.onClick = key1_click
81
82 ts2 = Touch(t_pin2)
83 ts2.onClick = key2_click
84
85 print("\nESP32 Touch Buttons & RGB Leds Demo")
86
87 #----- Για πάντα -----
88 while True:
89     ts1.checkButton() #Έλεγχος 1ου κουμπιού
90     ts2.checkButton() #Έλεγχος 2ου κουμπιού
91     time.sleep_ms(5) #Περίμενε 5 - 10msec

```

Ασκηση 4

Να κατασκευαστεί κλάση για τα κουμπιά αφής της πλακέτας η οποία ανιχνεύει μονό κλικ, διπλό κλικ και παρατεταμένο κλικ. Για κάθε συμβάν καλείται ξεχωριστή συνάρτηση εξυπηρέτησης. Επίσης να γραφεί κώδικας ο οποίος δημιουργεί στιγμιότυπα για δύο κουμπιά και εξυπηρετεί και τις τρεις λειτουργίες για το καθένα.

```

1 import machine, time
2 #----- Ορισμοί -----
3 threshold = 200 # Threshold to be adjusted
4 Debounce = 50 #msec
5 DbClickDelay = 300 #msec
6 LongPressDelay = 1000 #msec
7 t_pin1 = machine.TouchPad(machine.Pin(12)) #1ο κουμπί αφής
8 t_pin2 = machine.TouchPad(machine.Pin(13)) #2ο κουμπί αφής
9
10 #----- Ορισμός κλάσης -----
11 class Touch:
12     #Κατασκευαστής
13     def __init__(self, pin):
14         self.pin = pin
15         self.laststate = True
16         self.timer = 0
17         self.downtime = -1

```

```

18     self.uptime = -1
19     self.ignoreUP = False
20     self.singleClickOK = False
21     self.dblClickWaiting = False
22     self.dblClickOnNextUp = False
23     self.longPressHappened = False
24
25     #Συμβάντα
26     #Απλό κλικ
27     def onClick(self):
28         self()
29
30     #Διπλό κλικ
31     def onDblClick(self):
32         self()
33
34     #Παρατεταμένο κλικ
35     def onLongClick(self):
36         self()
37
38     #Καλείται περιοδικά και ελέγχει το κάθε κουμπί
39     def checkButton(self):
40         resultEvent = 0
41         self.timer = time.ticks_ms() #Κράτησε τον χρόνο συστήματος
42         cVal = self.pin.read() #Ανάβασε τιμή του αισθητήρα touch
43         state = False #Δεν πατήθηκε
44         if cVal < threshold: #Αν είναι κάτω από το κατώφλι
45             state = True #Πατήθηκε το κουμπί
46             #----- Πατημένο -----
47             #Πατήθηκε τώρα και πριν δεν είχε πατηθεί και ο uptime έχει ξεπεράσει τον χρόνο debounce.
48             #Άρχικά θα μπει εδώ αμέσως όταν πατηθεί γιατί δεν έχει κρατηθεί ο uptime
49             if state == True and self.laststate == False and (self.timer - self.uptime) > Debounce:
50                 self.downtime = self.timer #Άρχισε να μετράς τον χρόνο που είναι πατημένο - downtime
51                 self.ignoreUP = False
52                 self.singleClickOK = True
53                 self.longPressHappened = False
54                 #Αν δεν έχει περάσει ο χρόνος uptime τον χρόνο double click και δεν έχει ενεργοποιηθεί το double click στην επόμενη επαναφορά
55                 #και περιμένει για double click. Εδώ θα μπει κατά το 2ο πάτημα
56                 if (self.timer - self.uptime) < DblClickDelay and self.dblClickOnNextUp == False and self.dblClickWaiting == True:
57                     self.dblClickOnNextUp = True #Στο άφημα να το θεωρήσει διπλό κλικ
58                 else: #Εδώ θα μπει στο 1ο κλικ του διπλού κλικ
59                     self.dblClickOnNextUp = False
60                     self.dblClickWaiting = False #Επανάφορά για την επόμενη φορά
61             #----- Ελεύθερο -----
62             #Αλλιώς αν ήταν πατημένο και τώρα το άφησε και πέρασε ο downtime έχει ξεπεράσει τον χρόνο debounce
63             #Εδώ θα μπει αν κρατήθηκε πατημένο για χρόνο > debounce
64             elif state == False and self.laststate == True and (self.timer - self.downtime) > Debounce:
65                 #Αν δεν θέλει να αγνοήσει το Up. Εδώ δεν μπαίνει κατά το άφημα του long click
66                 if self.ignoreUP == False:
67                     self.uptime = self.timer #Άρχισε να μετράς τον χρόνο που είναι πάνω - uptime χρήσιμος για double click
68                     if self.dblClickOnNextUp == False: #Έχει γίνει το 1ο κλικ και τώρα το άφησε
69                         self.dblClickWaiting = True #Θα περιμένει για δεύτερο πάτημα
70                     #Εδώ έχει συμβεί διπλό κλικ. Είναι πάνω μετά από διπλό κλικ
71                 else:
72                     resultEvent = 2 #Αποτέλεσμα διπλό κλικ
73                     self.dblClickOnNextUp = False #Επανάφορά
74                     self.dblClickWaiting = False
75                     self.singleClickOK = False
76
77             #Ελέγχος για μονό κλικ. Ο χρόνος του διπλού κλικ έχει λήξει
78             if state == False and (self.timer - self.uptime) >= DblClickDelay and self.dblClickWaiting == True and self.dblClickOnNextUp == False and \
79                 self.singleClickOK == True and resultEvent != 2:
80                 resultEvent = 1
81                 self.dblClickWaiting = False
82
83             #Ελέγχος για παρατεταμένο κλικ
84             if state == True and (self.timer - self.downtime) >= LongPressDelay:
85                 #Πυροδότησε το παρατεταμένο πάτημα
86                 #Αν δεν πατήθηκε πριν παρατεταμένα
87                 if self.longPressHappened == False:
88                     resultEvent = 3
89                     self.ignoreUP = True #Αγνόησε το άφημα
90                     self.dblClickOnNextUp = False
91                     self.dblClickWaiting = False
92                     self.longPressHappened = True
93
94             self.laststate = state #Κράτα την προηγούμενη κατάσταση
95             #Κλήσεις ανάλογα με την ενέργεια
96             if resultEvent == 1:
97                 self.onClick()
98             if resultEvent == 2:
99                 self.onDblClick()
100             if resultEvent == 3:
101                 self.onLongClick()
102             #----- Τέλος ορισμού κλάσης -----
103
104             #----- Συναρτήσεις εξυπηρέτησης συμβάντων -----
105             def key1_click():
106                 print("key1 Click")
107
108             def key2_click():
109                 print("key2 Click")
110
111             def key1_dblclick():
112                 print("key1 Double Click")
113
114             def key2_dblclick():
115                 print("key2 Double Click")
116
117             def key1_longclick():
118                 print("key1 Long Click")
119
120             def key2_longclick():
121                 print("key2 Long Click")
122             #----- Τέλος συναρτήσεων εξυπηρέτησης συμβάντων -----
123
124             #===== Κυρίως πρόγραμμα =====
125             #----- Δημιουργία στιγμιотύπων και ορισμός συναρτήσεων εξυπηρέτησης -----
126             ts1 = Touch(t_pin1)
127             ts1.onClick = key1_click
128             ts1.onDblClick = key1_dblclick
129             ts1.onLongClick = key1_longclick
130
131             ts2 = Touch(t_pin2)
132             ts2.onClick = key2_click
133             ts2.onDblClick = key2_dblclick
134             ts2.onLongClick = key2_longclick
135
136             print("\nESP32 Touch Demo")
137
138             #----- Για πάντα -----
139             while True:
140                 ts1.checkButton() #Ελέγχος 1ου κουμπιού
141                 ts2.checkButton() #Ελέγχος 2ου κουμπιού
142                 time.sleep_ms(10) #Περίμενε 5 - 10msec

```

Ασκηση 5

Να γραφεί πρόγραμμα σε micropython το οποίο χρησιμοποιεί την ενσωματωμένη βιβλιοθήκη `asyncio` και αναβοσβήνει τα τρία από τα πέντε RGB Led, με σταθερά χρώματα το καθένα (κόκκινο, πράσινο, μπλε), με διαφορετική συχνότητα αναλαμπών. Συγκεκριμένα το πράσινο αναβοσβήνει με περίοδο 4 sec, το κόκκινο με περίοδο 1 sec και το μπλε με περίοδο 0,2 sec. Η `asyncio` επιτρέπει την ασύγχρονη εκτέλεση των προγραμμάτων χωρίς την χρήση εντολών blocking όπως η `time.sleep()`.

```

1 import machine, neopixel, asyncio
2
3 # Προετοιμασία RGB Led
4 np = neopixel.NeoPixel(machine.Pin(25), 5) #IO25, 5 x RGB Leds
5 led_status = [0, 0, 0, 0, 0] #Λίστα με την κατάσταση των Led (0=σβηστό, 1=αναμμένο)
6 RED = (255, 0, 0) #Κόκκινο
7 GREEN = (0, 255, 0) #Πράσινο
8 BLUE = (0, 0, 255) #Μπλε
9
10 # Η συνάρτηση αναβοσβήνει το Led με αριθμό num (0-4) με το χρώμα color
11 def toggle_led(num, color):
12     if led_status[num] == 0: #Είναι σβηστό
13         np[num] = color #Γράψε χρώμα
14         led_status[num] = 1 #Σημείωσε ότι τώρα είναι αναμμένο
15     else: #Διαφορετικά είναι αναμμένο
16         np[num] = (0, 0, 0) #Σβήσε (0,0,0) = σβηστό (μαύρο)
17         led_status[num] = 0 #Σημείωσε ότι τώρα είναι σβηστό
18     np.write() #Στείλε εντολή στα LED
19
20 # Ασύγχρονη λειτουργία (coroutine) για Πράσινο
21 async def blink_green_led():
22     while True: #Για πάντα
23         toggle_led(4, GREEN) #Αναβόσβησε το 1ο στην σειρά με χρώμα πράσινο
24         await asyncio.sleep(2) #Περίμενε 2 sec και δώσε την δυνατότητα εκτέλεσης άλλων λειτουργιών
25
26 # Ασύγχρονη λειτουργία (coroutine) για Κόκκινο
27 async def blink_red_led():
28     while True:
29         toggle_led(0, RED) #Αναβόσβησε το 5ο στην σειρά με χρώμα κόκκινο
30         await asyncio.sleep(0.5) #Περίμενε .5 sec και δώσε την δυνατότητα εκτέλεσης άλλων λειτουργιών
31
32 # Ασύγχρονη λειτουργία (coroutine) για Μπλε
33 async def blink_blue_led():
34     while True: #Για πάντα
35         toggle_led(2, BLUE) #Αναβόσβησε το μεσαίο με χρώμα μπλε
36         await asyncio.sleep(0.1) #Περίμενε 100 msec και δώσε την δυνατότητα εκτέλεσης άλλων λειτουργιών
37
38 # Ορισμός της συνάρτησης main που περιέχει το event loop
39 async def main():
40     # Δημιουργία εργασιών (tasks) ώστε να αναβοσβήνουν τα 3 Led ταυτόχρονα
41     asyncio.create_task(blink_green_led())
42     asyncio.create_task(blink_red_led())
43     asyncio.create_task(blink_blue_led())
44
45 # Δημιουργία και εκτέλεση του event loop
46 loop = asyncio.get_event_loop()
47 loop.create_task(main()) # Δημιουργία εργασίας (task) που εκτελεί το event loop
48 loop.run_forever() # Τρέξε για πάντα

```

Ασκηση 6

Να γραφεί πρόγραμμα σε micropython το οποίο διαβάζει την θέση του ποτενσιομέτρου του αρθρώματος επέκτασης AJS06.

Συνδέουμε το ποτενσιόμετρο AJS06 σε οποιαδήποτε από τις τέσσερις πάνω θύρες της πλακέτας (IO4, IO26, IO32, IO34) με το τροποποιημένο καλώδιο ή δύο EXP-AJ11 όπου το ένα έχει τροποποιηθεί και την breadboard. Ο μικροελεγκτής ESP32 διαθέτει μετατροπέα από αναλογικό σε ψηφιακό (Analog to Digital Converter **ADC**) εύρους 12bit δηλαδή το εύρος τιμών είναι από 0-4095. Μπορούμε να μειώσουμε την ευκρίνεια σε 11, 10 ή 9 bit. Επίσης διαθέτει στην είσοδο μεταβλητό εξασθενητή με τον οποίο αλλάζουμε το εύρος τάσεων που μπορεί να μετατρέψει. Δηλαδή αν επιλέξουμε την σταθερά `ADC.ATTN_6DB` τότε για τάση εισόδου 2 V ο ADC θα επιστρέψει τιμή 4095 στα 12bit.

ADC.ATTN_0DB - εύρος τιμών έως 1.2V

ADC.ATTN_2_5DB - εύρος τιμών έως 1.5V

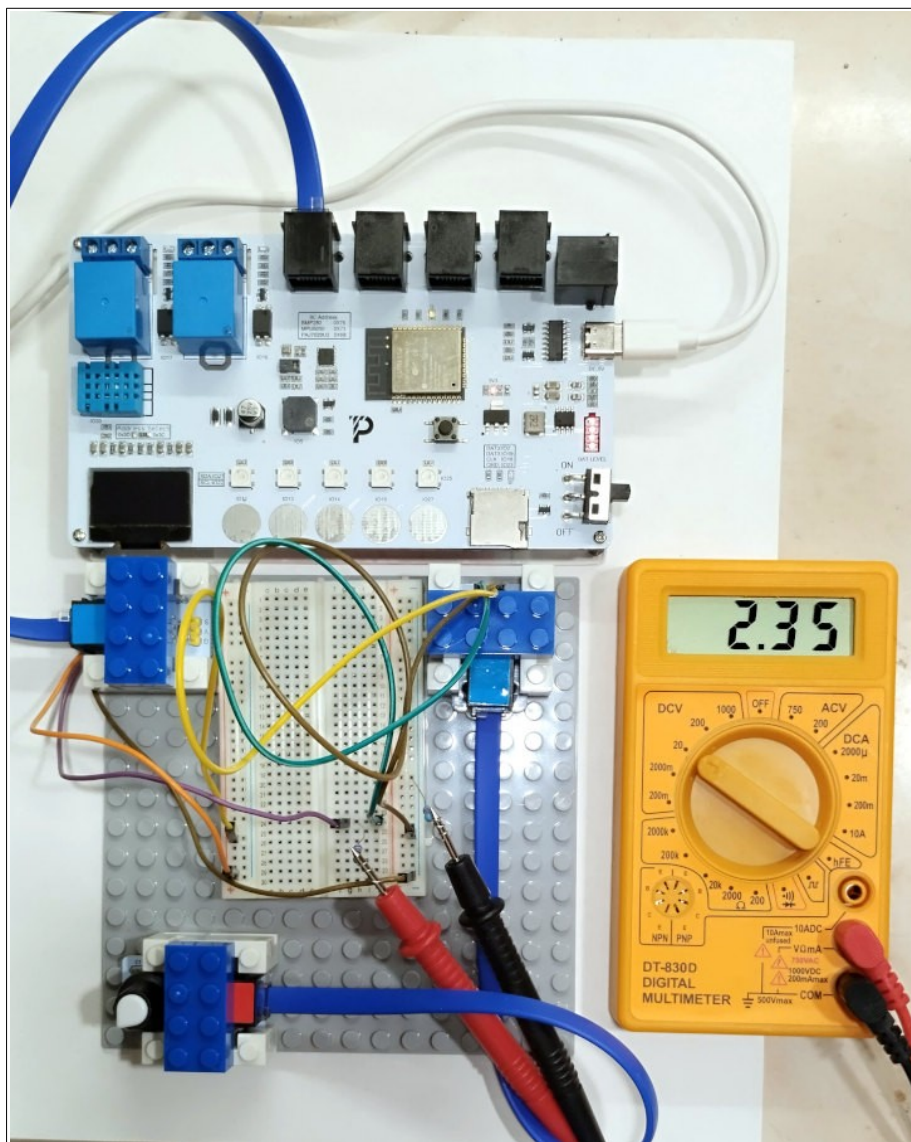
ADC.ATTN_6DB - εύρος τιμών έως 2.0V

ADC.ATTN_11DB - εύρος τιμών έως 3.3V

Επειδή το ένα άκρο του ποτενσιομέτρου συνδέεται στα 3,3V και το άλλο στην γείωση (GND), επιλέγουμε την σταθερά `ADC.ATTN_11DB`. Ο δρομέας οδηγείται στην είσοδο του ADC.

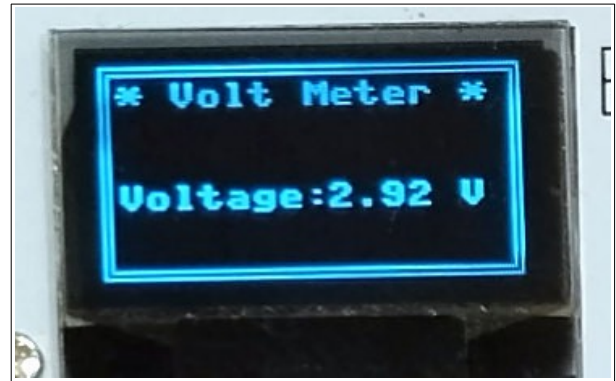
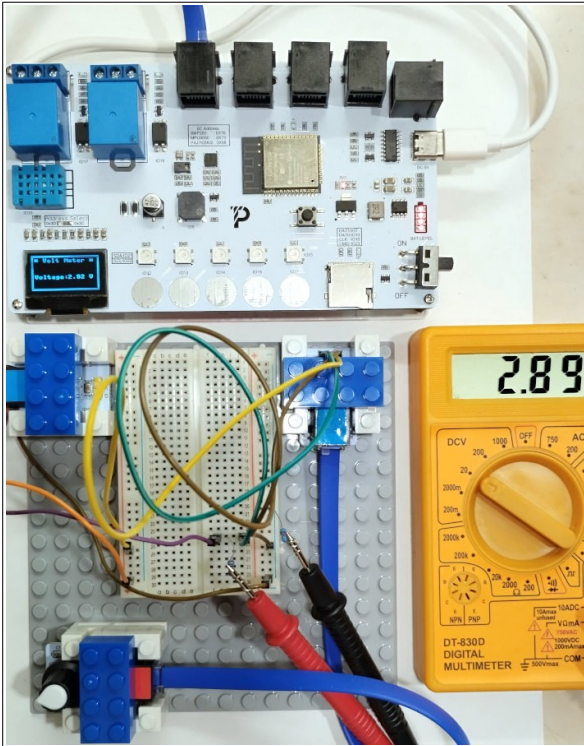
```
1 from machine import Pin, ADC
2 from time import sleep
3
4 pot = ADC(Pin(4)) #4, 26, 32, 34
5 pot.width(ADC.WIDTH_12BIT) #Ευκρίνεια 12BIT προκαθορισμένο, 9BIT, 10BIT, 11BIT
6 pot.atten(ADC.ATTN_11DB) # πλήρης κλίμακα έως 3.3V
7
8 while True:
9     pot_value = pot.read()
10    voltage = pot_value * (3.3 / 4096) #12bit
11    print("τιμή:", pot_value, "τάση:", round(voltage, 2), "V")
12    sleep(0.5)
```

Στην breadboard μπορούμε να συνδέσουμε και το πολύμετρο σε λειτουργία βολτομέτρου (μαύρος ακροδέκτης στο '-' και κόκκινος στον δρομέα ο οποίος συνδέεται με την είσοδο ADC. Για ευκολία μπορούμε να τυλίξουμε στον κάθε ακροδέκτη έναν αντιστάτη 220Ω και το άλλο άκρο να καρφωθεί στην breadboard. Εκτελούμε τον κώδικα και περιστρέφοντας αργά το ποτενσιόμετρο παρατηρούμε αν οι τιμές στο βολτόμετρο συμφωνούν με αυτές στο τερματικό του vscode. Φυσιολογικά πρέπει να υπάρχουν αποκλίσεις της τάξεως των 10 – 20mV οι οποίες οφείλονται σε σφάλμα του πολυμέτρου αλλά κυρίως στη μη γραμμική συμπεριφορά του ADC. Αυτό μπορεί να διορθωθεί με πίνακες διόρθωσης στον κώδικα και βαθμονόμηση (καλιμπράρισμα) με κάποιο πιο αξιόπιστο βολτόμετρο.



Άσκηση 7

Να τροποποιηθεί το παραπάνω ώστε να εμφανίζεται στην ενσωματωμένη μονόχρωμη οθόνη OLED η τιμή της τάσης σε Volts.



Στον κώδικα παρατηρούμε ότι έχει προστεθεί η μεταβλητή **avg_voltage** στην γραμμή 7. Αυτή είναι η μέση τιμή των μετρήσεων του ADC. Με την χρήση μέσης τιμής αποφεύγουμε την συνεχόμενη μεταβολή σε κάθε μέτρηση. Δηλαδή έχουμε μια εξομάλυνση στις ενδείξεις σαν να είχαμε συνδέσει παράλληλα στην είσοδο του ADC έναν πυκνωτή. Επίσης προσθέσαμε μια σταθερά **OFFS_COEF** που σαν σκοπό έχει να βελτιώσει την απόκλιση των

μετρήσεων σε σχέση με το φυσικό ψηφιακό βολτόμετρο. Η βελτίωση είναι μερική γιατί με τον συντελεστή (συνάρτηση 1ου βαθμού) δεν διορθώνονται μη γραμμικές συμπεριφορές στα άκρα. Στις παραπάνω εικόνες βλέπουμε την τιμή που εμφανίζει η οθόνη μας σε σχέση με αυτή του βολτομέτρου.

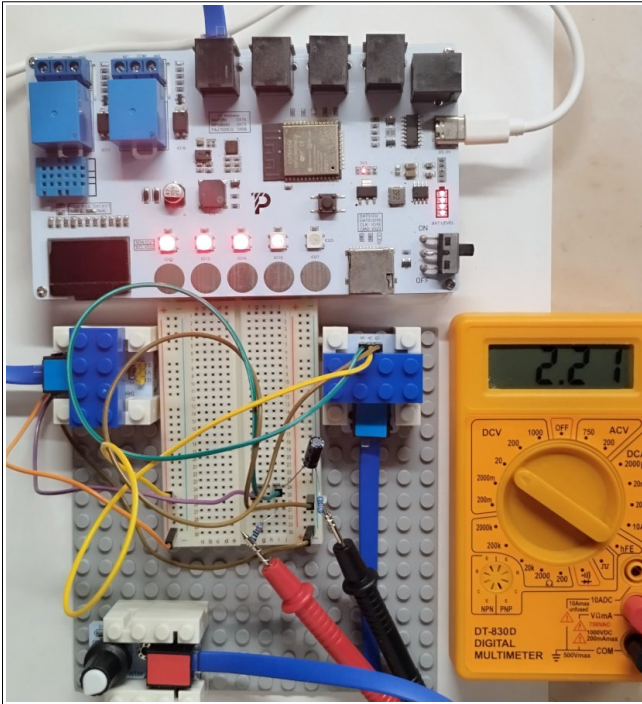
```

1 from machine import Pin, ADC, I2C
2 from time import sleep
3 from ssd1306 import SSD1306_I2C
4
5 AVG_FACTOR = 2.0 #Συντελεστής μέσης τιμής όσο μεγαλύτερος τόσο πιο αργή μεταβολή
6 OFFS_COEF = .95 #Συντελεστής διόρθωσης
7 avg_voltage = 0
8
9 pot = ADC(Pin(4)) #4, 26, 32, 34
10 pot.width(ADC.WIDTH_12BIT) #Ευκρίνεια 12BIT προκαθορισμένο, 9BIT, 10BIT, 11BIT
11 pot.atten(ADC.ATTN_11DB) # πλήρης κλίμακα έως 3.3V
12
13 i2c = I2C(0, sda = Pin(21), scl = Pin(22), freq = 400000)
14 display = SSD1306_I2C( width=128, height=64, i2c=i2c, addr=0x3c, external_vcc=False )
15 display.fill(0) #Καθαρίζει οθόνη
16 #Δημιουργεί ετικέτες
17 display.text('* Volt Meter *', 5, 5, 1) #θέση x, y, color 1/0
18 display.text("Voltage: ", 5, 35, 1)
19 display.text("V", 108, 35, 1)
20 #Δημιουργεί διπλό περίγραμμα
21 display.rect(0,0,127,63,1,0) #x, y, width, height, color=0/1, fill=0/1
22 display.rect(2,2,123,59,1,0) #x, y, width, height, color=0/1, fill=0/1
23 display.show() #Παρουσίαση στην οθόνη
24
25 while True:
26     pot_value = pot.read()
27     voltage = pot_value * (3.3 / 4096) / OFFS_COEF #12bit Στιγματική τιμή
28     avg_voltage = (AVG_FACTOR * avg_voltage + voltage) / (AVG_FACTOR + 1) #Μέση τιμή
29     print("τιμή:", pot_value, "τάση:", round(voltage, 2), "V")
30     display.rect(68, 34, 40, 9, 0, True) #Σβήσε παλιά τιμή
31     display.text(str(round(avg_voltage, 2)), 68, 35, 1) #Εμφάνισε νέα τιμή
32     display.show() #Παρουσίαση στην οθόνη
33     sleep(0.5)

```

Άσκηση 8

Να τροποποιηθεί η άσκηση 6 ώστε με την περιστροφή του ποτενσιομέτρου να ανάβουν διαδοχικά τα 5 RGB Leds της πλακέτας σε κόκκινο χρώμα. Το τελευταίο αναμμένο LED θα ανάβει αναλογικά δηλαδή αν έχουν ανάψει πλήρως δύο LED και η τιμή του ADC δεν επαρκεί ώστε να ανάψει πλήρως και τρίτο LED τότε αυτό θα ανάψει με περιορισμένη ένταση π.χ. 30%.



Παρατηρήστε ότι το τέταρτο LED ανάβει με χαμηλότερη ένταση από τα τρία αριστερά του. Η ευκρίνεια του ADC έχει μειωθεί στα 10 bits.

Για να μειωθεί το 'παίξιμο' των τιμών κατά την δειγματοληψία του ADC βάλαμε έναν ηλεκτρολυτικό πυκνωτή 100μF από τον δρομέα στην γείωση (GND). Ο δρομέας συνδέεται στην είσοδο του ADC (IO4). Το μακρύ ποδαράκι του πυκνωτή είναι το (+) και συνδέεται στον δρομέα, ενώ το κοντό είναι το (-) και συνδέεται στην γείωση (- ή GND).

```

1 from machine import Pin, ADC
2 from time import sleep
3 from neopixel import NeoPixel
4
5 RED = (255, 0, 0); GREEN = (0, 255, 0); BLUE = (0, 0, 255); BLANK = (0, 0, 0)
6 AVG_FACTOR = 3 # Συντελεστής μέσης τιμής όσο μεγαλύτερος τόσο πιο αργή μεταβολή
7 avg_val = 0
8
9 pot = ADC(Pin(4)) #4, 26, 32, 34
10 pot.width(ADC.WIDTH_10BIT) # Ευκρίνεια 10BIT, 9BIT, 10BIT, 11BIT, 12BIT προκαθορισμένο
11 pot.atten(ADC.ATTN_11DB) # πλήρης κλίμακα έως 3.3V
12
13 np = NeoPixel(Pin(25), 5) #IO25, 5 x RGB Leds
14
15 # Συνάρτηση παρόμοια με την map του Arduino
16 # Δέχεται τρέχουσα τιμή, ελάχιστη τιμή, μέγιστη τιμή, έξοδος από, έως
17 def map(value, in_min, in_max, out_min, out_max):
18     scaled_value = (value - in_min) * (out_max - out_min) / (in_max - in_min) + out_min
19     return int(scaled_value) # Επιστρέφει ακέραιο
20
21 # Συνάρτηση για να ανάβει τα Leds ανάλογα με την τιμή του ADC
22 def out_leds(x):
23     n = x // 205 # Γιατί 1024 / 5 leds = 204,8 ακέραια διαίρεση
24     y = x % 205 # Υπόλοιπο για το τελευταίο Led
25     i = 0
26     while i < n: # Ανάβει πλήρως όσα Led υπολογίστηκαν στο n
27         np[4 - i] = RED # Ανάβουν ανάποδα
28         i += 1
29     a = i # Κράτησε την τρέχουσα τιμή του i
30     while i < 5: # Αν δεν είναι να ανάψουν όλα τότε σβήσε τα υπόλοιπα
31         np[4 - i] = BLANK # Σβήνουν ανάποδα
32         i += 1
33     print(n, y, map(y, 0, 204, 2, 255))
34     i = a # Επαναφορά του i
35     if i <= 4: # Αν είναι από 0 έως 4 τότε άναψε μερικώς αυτό το Led
36         np[4 - i] = (map(y, 0, 204, 2, 250), 0, 0) # Αλλαγή κλίμακας από 0-204 σε 2-255 για το κόκκινο (255 πλήρης ένταση)
37     np.write()
38
39 # Για πάντα
40 while True:
41     pot_value = pot.read() # Διάβασε τιμή ποτενσιομέτρου
42     avg_val = int((AVG_FACTOR * avg_val + pot_value) / (AVG_FACTOR + 1)) # Μέση τιμή
43     out_leds(avg_val) # Άναψε τα Leds
44     print("τιμή:", avg_val)
45     sleep(.1)

```

Ασκηση 9

Να τροποποιηθεί ο παραπάνω κώδικας ώστε να χρησιμοποιηθεί το ποτενσιόμετρο ως συσκευή εισόδου εντολών. Συγκεκριμένα ας υποθέσουμε ότι θέλουμε να επιλέγουμε πέντε προγράμματα πλυντηρίου περιστρέφοντας το ποτενσιόμετρο και ανάλογα την θέση να εμφανίζεται το πρόγραμμα στο τερματικό π.χ. 'Θέση-2' και να ανάβει ένα από τα 5 RGB Leds με διαφορετικό χρώμα. Για τον συγκεκριμένο σκοπό υπάρχουν άλλα εξαρτήματα, όπως περιστροφικοί διακόπτες 16 θέσεων που βγάζουν 4bit ανάλογα την θέση ή R.I.E. (Rotary Impulse Encoders) που καταλαβαίνουν πόσα βήματα έχουν περιστραφεί και προς ποια κατεύθυνση. Επειδή το kit δεν διαθέτει κάτι αντίστοιχο εμείς θα χρησιμοποιήσουμε το ποτενσιόμετρο.

```

1 from machine import Pin, ADC
2 from time import sleep
3 from neopixel import NeoPixel
4
5 #Σταθερές χρωμάτων
6 RED = (255, 0, 0); GREEN = (0, 255, 0); BLUE = (0, 0, 255); YELLOW = (255, 255, 0)
7 MAGENTA = (255, 0, 255); CYAN = (0, 255, 255); BLANK = (0, 0, 0)
8 AVG_FACTOR = 1.5 #Συντελεστής μέσης τιμής όσο μεγαλύτερος τόσο πιο αργή μεταβολή
9 avg_val = 0 #Μέση τιμή του ADC
10
11 pot = ADC(Pin(4)) #4, 26, 32, 34
12 pot.width(ADC.WIDTH_10BIT) #Ευκρίνεια 10BIT, 9BIT, 10BIT, 11BIT, 12BIT προκαθορισμένο
13 pot.atten(ADC.ATTN_11DB) # πλήρης κλίμακα έως 3.3V
14
15 np = NeoPixel(Pin(25), 5) #IO25, 5 x RGB Leds
16
17 #Λίστα με τις εντολές
18 C = ['Θέση - 1', 'Θέση - 2', 'Θέση - 3', 'Θέση - 4', 'Θέση - 5']
19 #Παράλληλη λίστα με τα χρώματα για κάθε θέση
20 COLORS = [RED, GREEN, BLUE, YELLOW, MAGENTA]
21 PLACES = len(C) #Υπολόγισε αριθμό θέσεων
22 THRESS = 5 #Κατώφλι υστέρησης
23 pos = 0 #Θέση στην λίστα με τις εντολές
24
25 #Η συνάρτηση σβήνει και τα 5 RGB Leds
26 def blank_leds():
27     for i in range(5):
28         np[i] = BLANK
29
30 #Συνάρτηση για να ανάβει τα Leds ανάλογα με την τιμή του ADC
31 def out_leds(n):
32     blank_leds()
33     np[4 - n] = COLORS[n]
34     np.write()
35
36 #Η συνάρτηση επιλέγει εντολή ανάλογα με την τιμή του x δηλαδή την θέση του ποτενσιόμετρου
37 #Υπάρχει πρόβλεψη υστέρησης τιμής THRESS ώστε να μην παίζει η απόφαση της εντολής σε γειτονικές
38 #τιμές. π.χ. αν είναι 127 και 128 τότε θα μεταβάλλεται από θέση-1 σε θέση-2. Με υστέρηση π.χ. 5
39 #για να αλλάξει σε θέση-1 πρέπει να πέσει κάτω από 123 και για να ανέβει σε θέση-2 πρέπει να
40 #ξεπεράσει το 133.
41 def get_command(x):
42     global pos
43     stp = 1024 // PLACES #Βήμα για κάθε θέση
44     s = 0 #Αθροισμα βημάτων
45     i = 0
46     for i in range(8):
47         if x > s + THRESS and x < s + stp - THRESS: #Αν ξέφυγε από την περιοχή υστέρησης τότε
48             pos = i #Αλλάξε την τιμή του pos, διαφορετικά ισχύει η προηγούμενη τιμή
49             s += stp
50     return pos
51
52 #Για πάντα
53 while True:
54     pot_value = pot.read() #Διάβασε τιμή ποτενσιόμετρου
55     avg_val = int((AVG_FACTOR * avg_val + pot_value) / (AVG_FACTOR + 1)) #Μέση τιμή
56     tmp = get_command(avg_val)
57     print(C[tmp]) #Εμφάνισε εντολή
58     out_leds(tmp)
59     #print("τιμή:", avg_val) #Debug
60     sleep(1)

```

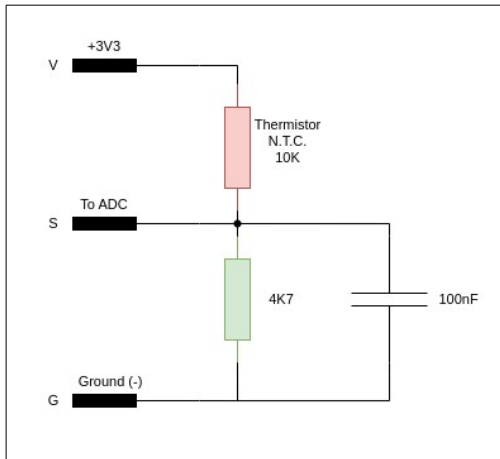
Ασκηση 10

Μέτρηση θερμοκρασίας με αναλογικό αισθητήρα. Στα προηγούμενα χρησιμοποιήσαμε για μέτρηση θερμοκρασίας, ψηφιακούς αισθητήρες όπως ο DHT11 ή ο BMP280. Αυτοί έχουν βαθμονομηθεί από τις κατασκευάστριες εταιρείες και στέλνουν τις μετρήσεις ψηφιακά μέσω διαύλων όπως IIC, onewire, SPI κλπ. και ο προγραμματιστής μπορεί να εμπιστευτεί την μέτρηση ως αξιόπιστη και να την χρησιμοποιήσει στην εφαρμογή του.

Μπορούμε αν θέλουμε να χρησιμοποιήσουμε τον αισθητήρα **AJS01** ο οποίος διαθέτει έναν ειδικό αντιστάτη το **thermistor** στον οποίο μεταβάλλεται κατά πολύ η αντίσταση με τις μεταβολές της θερμοκρασίας. Σε θερμοκρασία 25°C η αντίσταση για τον συγκεκριμένο είναι 10K. Όταν η

θερμοκρασία ανέβει η αντίσταση θα μειωθεί και όταν κατέβει θα αυξηθεί. Δηλαδή έχει αρνητικό συντελεστή θερμοκρασίας (**N.T.C.** Negative Temperature Coefficient).

Το άρθρωμα **AJS01** έχει την ακόλουθη απλή συνδεσμολογία. Δηλαδή έχουμε έναν διαιρέτη τάσης με το thermistor NTC και έναν αντιστάτη γνωστής αντίστασης. Το κοινό σημείο πηγαίνει στον



μετατροπέα ADC και μετρώντας την τάση υπολογίζουμε την αντίσταση του NTC.

Στη συνέχεια εφαρμόζοντας κάποιους τύπους με τους τρεις συντελεστές που βρίσκουμε στο φύλλο δεδομένων του εξαρτήματος μπορούμε να υπολογίσουμε την θερμοκρασία. Η χρήση thermistor είναι οικονομική και απλή αλλά δεν έχουμε μεγάλη ακρίβεια και χρειάζεται βαθμονόμηση με κάποιο θερμόμετρο αναφοράς. Συνήθως χρησιμοποιείται σε εφαρμογές στις οποίες δεν μας ενδιαφέρει τόσο πολύ η ακρίβεια ($\pm 2^\circ\text{C}$) όπως θερμοστάτες συσκευών κλπ. Επιπλέον είναι ανθεκτικός σε πολύ υψηλές θερμοκρασίες $>100^\circ\text{C}$ όπου οι ψηφιακοί αισθητήρες παρουσιάζουν προβλήματα.

Συνδέουμε το άρθρωμα AJS01 με τροποποιημένο καλώδιο στην θύρα IO4 και δοκιμάζουμε τον ακόλουθο κώδικα. Για σύγκριση χρησιμοποιούμε και τον ενσωματωμένο αισθητήρα DHT11.

```
1 from machine import Pin, ADC
2 from dht import DHT11
3 from time import sleep
4 import math
5
6 R2 = 4660 #Ω
7 Vi = 3.27 #Volts
8
9 #Σταθερές Steinhart για το NTC Thermistor TCS610
10 A = 1.1279e-03 #0.001129148
11 B = 2.3429e-04 #0.000234125
12 C = 8.7298E-08 #0.000000876741
13
14 ntc = ADC(Pin(4)) #4, 26, 32, 34
15 ntc.width(ADC.WIDTH_12BIT) #Ευκρίνεια 10BIT, 9BIT, 10BIT, 11BIT, 12BIT προκαθορισμένο
16 ntc.atten(ADC.ATTN_11DB) # πλήρης κλίμακα έως 3.3V
17
18 sensor = DHT11(Pin(33)) #DHT11 για σύγκριση
19 sensor.measure() #Νέα μέτρηση του DHT11
20
21 #Για πάντα
22 while True:
23     vo = ntc.read() #Διάβασε τιμή της τάσης του διαιρέτη
24     vo /= .9 #Διόρθωση μη γραμμικής συμπεριφοράς ADC
25     Rntc = R2 * (4095.0 / vo - 1); #Υπολογισμός αντίστασης NTC
26     #Υπολογισμός θερμοκρασίας σε βαθμούς Kelvin
27     TempK = 1 / (A + (B * math.log(Rntc)) + C * math.pow(math.log(Rntc), 3))
28     TempC = TempK - 273.15 #Μετατροπή σε °C
29     print("Θερμοκρασία NTC: ", round(TempC, 2), "°C") #Εμφάνισε θερμοκρασία NTC
30     print("Θερμοκρασία DHT11: ", sensor.temperature() + 1.0, "°C\n")
31     sleep(2) #Περίμενε 2 sec
```

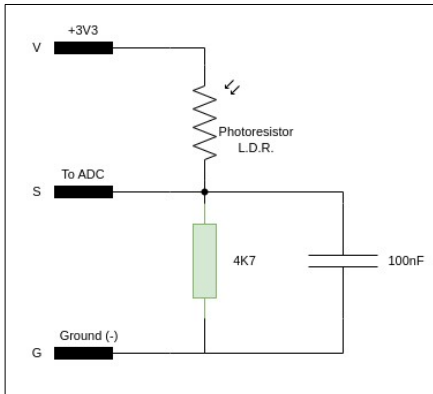
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
>>>
MPY: soft reboot
Θερμοκρασία NTC: 26.57 °C
Θερμοκρασία DHT11: 26.0 °C

Θερμοκρασία NTC: 26.52 °C
Θερμοκρασία DHT11: 26.0 °C
```

Άσκηση 11

Αισθητήρας φωτός – Φωτοαντιστάτης LDR. Πολλές φορές χρειάζεται να φτιάξουμε αυτοματισμούς οι οποίοι με δεδομένο την ένταση του ορατού φωτός, κάνουν κάποιες ενέργειες π.χ. ανάβουν ή σβήνουν τα φώτα.

Να γίνει κύκλωμα και πρόγραμμα σε Micropython με τα αρθρώματα DJX01 (λευκό LED) και AJS03 (φωτοαντιστάτης LDR), ώστε αν η ένταση του φωτός μειωθεί να ανάβει το LED και όταν είναι αρκετή να το σβήνει.



Δίπλα φαίνεται το κύκλωμα του αρθρώματος AJS03. Όπως καταλαβαίνουμε πρόκειται για αναλογική μεταβολή της τάσης, επομένως θα συνδεθεί σε είσοδο ADC.

Συνδέουμε με τροποποιημένα καλώδια RG12 το LDR στην υποδοχή IO4 και το LED στην IO32.

Στη συνέχεια δοκιμάζουμε το παρακάτω πρόγραμμα. Τροποποιείτε τις τιμές στις γραμμές 17 και 19 ώστε το πρόγραμμα να λειτουργεί σωστά στο περιβάλλον του εργαστηρίου.

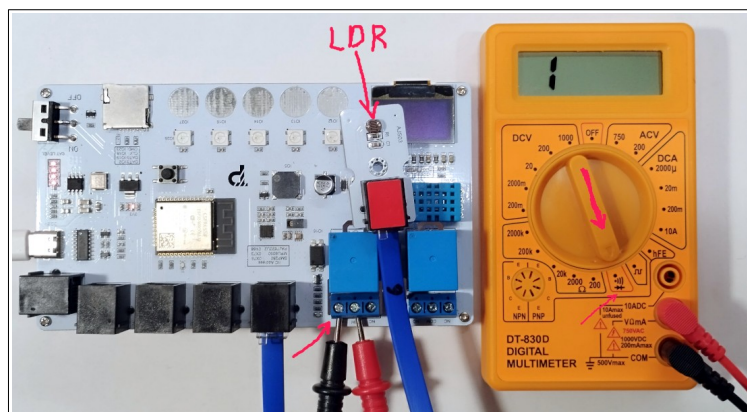
```

1 from machine import Pin, ADC
2 from time import sleep
3
4 #Αντίσταση LDR από 600K στο σκοτάδι έως 200Ω σε δυνατό φως
5 ldr = ADC(Pin(4)) #4, 26, 32, 34
6 ldr.width(ADC.WIDTH_12BIT) #Ευκρίνεια 10BIT, 9BIT, 10BIT, 11BIT, 12BIT προκαθορισμένο
7 ldr.atten(ADC.ATTN_11DB) # πλήρης κλίμακα έως 3.3V
8 #LED DJX01 στο pin IO32
9 light = Pin(32, Pin.OUT) #Ψηφιακή έξοδος
10 light.value(0) #Σβηστό
11
12 #Για πάντα
13 while True:
14     vo = ldr.read() #Διάβασε τιμή της τάσης του διαιρέτη
15     #Οι τιμές είναι από 0 (απόλυτο σκοτάδι) έως 4095 (δυνατό φως)
16     print("Τιμή LDR: ", vo) #Εμφάνισε τιμή ADC
17     if vo < 500: #Δυνατότητα υστέρησης για αποφυγή αναλαμπών
18         light.value(1) #Ανάψε το LED
19     elif vo > 800:
20         light.value(0) #Σβήσε το LED
21     sleep(2) #Περίμενε 2 sec

```

Να τροποποιηθεί το παραπάνω ώστε να ανάβει τα φώτα σε δίκτυο 220V μέσω του ηλεκτρονόμου IO16.

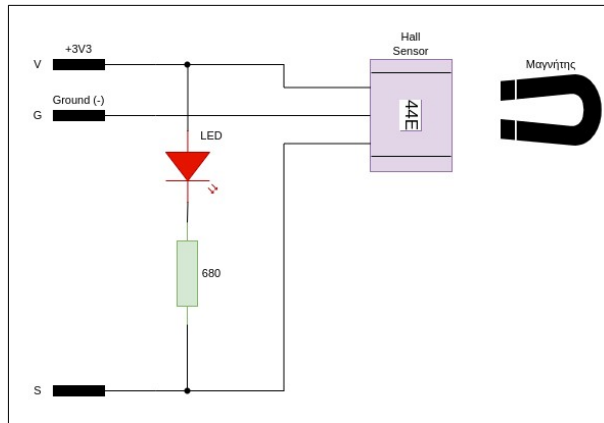
Εδώ η μοναδική αλλαγή γίνεται στην γραμμή 9 όπου το 32 θα γίνει 16. Γυρίστε τον μεταγωγό του πολυμέτρου στην θέση με το **beep** – **δίοδο** και συνδέστε τους ακροδέκτες στην κλέμα στα σημεία COM και ON. Ελέγξτε αν στο σκοτάδι κλείνει ο ηλεκτρονόμος και ακούγεται σφύριγμα από το πολύμετρο.



Άσκηση 12

Αισθητήρας Hall. Ο αισθητήρας Hall είναι ένα εξάρτημα ανίχνευσης μαγνητικού πεδίου. Όταν ένας μαγνήτης πλησιάσει στον αισθητήρα αυτός αλλάζει την κατάσταση του. Το συγκεκριμένο άρθρωμα του kit βγάζει στον ακροδέκτη S λογικό '1' όταν δεν υπάρχει μαγνητικό πεδίο και λογικό '0' όταν πλησιάσει κοντά ένας μικρός μαγνήτης. Χρησιμοποιείται ευρέως σε ηλεκτρικές συσκευές, αυτοκίνητα κ.λ.π.

Να γίνει κύκλωμα και πρόγραμμα σε Micropython με τα αρθρώματα DJX01 (λευκό LED) και DJS07 (αισθητήρα Hall), ώστε όταν ο μαγνήτης πλησιάζει να ανάβει το LED και όταν απομακρύνεται να σβήνει.



```
1 from machine import Pin
2 from time import sleep
3
4 #Αισθητήρας Hall DJS07 στο pin IO4
5 hall = Pin(4, Pin.IN, Pin.PULL_UP)
6 #LED DJX01 στο pin IO32
7 light = Pin(32, Pin.OUT) #Ψηφιακή έξοδος
8 light.value(0) #Σβηστό
9
10 #Για πάντα
11 while True:
12     state = hall.value()
13     print("Κατάσταση Hall: ", state) #Εμφάνισε κατάσταση αισθητήρα Hall
14     if not state: #Αν είναι 0 έχει πλησιάσει ο μαγνήτης
15         light.value(1) #Ανάψε το LED
16     else: #Διαφορετικά δεν υπάρχει ικανό μαγνητικό πεδίο
17         light.value(0) #Σβήσε το LED
18         sleep(1) #Περίμενε 1 sec
```

Άσκηση 13

Επιλέγον γραμματοσειρές στην οθόνη OLED.

Αρχικά κατεβάζουμε από το [github](https://github.com) το πρόγραμμα **font_to_py.py**. Έπειτα πάμε στον ιστότοπο <https://www.fontspace.com/category/ttf> και κατεβάζουμε ένα οποιοδήποτε font π.χ. Ariana Violeta. Αποσυμπιέζουμε το αρχείο και αν θέλουμε βλέπουμε το δείγμα με την εφαρμογή προβολής γραμματοσειρών του λειτουργικού μας συστήματος. Αντιγράφουμε το αρχείο .ttf στον ίδιο φάκελο με το πρόγραμμα font_to_py.py. Για συστήματα Linux φτιάχνουμε ένα **venv** π.χ. `python3 -m venv ~/micropython`. Χρησιμοποιούμε το περιβάλλον με `source ~/micropython/bin/activate`. Εγκαθιστούμε το argpass με `pip install argpass` και την βιβλιοθήκη freetype-py με `pip install freetype-py`. Για συστήματα Windows απλώς `pip install argpass` χωρίς την χρήση venv.

Στη συνέχεια μεταφράζουμε το αρχείο από ttf σε py με την εντολή:

```
python font_to_py.py ArianaVioleta.ttf 32 ArianaVioleta_python.py
```

Το 32 είναι μέγεθος και με δοκιμές επιλέγουμε αυτό που μας ταιριάζει.

Μέσα στον φάκελο εκτός από το boot.py και main.py πρέπει να συμπεριληφθούν τα αρχεία ssd1306.py, writer.py, η γραμματοσειρά ArianaVioleta_python.py και οποιαδήποτε άλλη γραμματοσειρά.

```
1 from machine import Pin, I2C
2 from ssd1306 import SSD1306_I2C
3 from writer import Writer
4 import ArianaVioleta_python #Μεταφρασμένη γραμματοσειρά
5
6 i2c = I2C(0, sda = Pin(21), scl = Pin(22), freq = 400000)
7 display = SSD1306_I2C( width=128, height=64, i2c=i2c, addr=0x3c, external_vcc=False )
8 display.fill(0) #Καθαρίζει οθόνη
9
10 wri = Writer(display, ArianaVioleta_python) #Χρήση γραμματοσειράς
11 wri.set_textpos(display, 0, 0) #Αρχική θέση y, x
12 wri.printstring("Hello\n") #Εμφάνιση κειμένου
13 wri.printstring("World")
14 display.text("Test", 90, 56) #Εμφάνιση κειμένου με τον κλασικό τρόπο
15 display.show() #Παρουσίασε στην οθόνη
```

Αν θέλουμε μπορούμε να μεταφράσουμε γραμματοσειρές από τον υπολογιστή μας. Για παράδειγμα στο Ubuntu Linux οι γραμματοσειρές βρίσκονται στο /usr/share/fonts/truetype.

Στον παρακάτω κώδικα χρησιμοποιούμε δύο γραμματοσειρές ταυτόχρονα.



```
1 from machine import Pin, I2C
2 from ssd1306 import SSD1306_I2C
3 from writer import Writer
4 import ArianaVioleta_python, FreeSerifBold_python #Μεταφρασμένες γραμματοσειρές
5
6 i2c = I2C(0, sda = Pin(21), scl = Pin(22), freq = 400000)
7 display = SSD1306_I2C( width=128, height=64, i2c=i2c, addr=0x3c, external_vcc=False )
8 display.fill(0) #Καθαρίζει οθόνη
9
10 wri1 = Writer(display, ArianaVioleta_python) #Χρήση γραμματοσειράς στο wri1
11 wri2 = Writer(display, FreeSerifBold_python) #Χρήση γραμματοσειράς στο wri2
12 wri1.set_textpos(display, 0, 0) #Αρχική θέση y, x
13 wri1.printstring("Hello") #Εμφάνιση κειμένου
14 wri2.set_textpos(display, 30, 0) #Αρχική θέση y, x
15 wri2.printstring("World")
16 display.text("Test", 90, 56) #Εμφάνιση κειμένου με τον κλασικό τρόπο στην θέση x, y
17 display.show() #Παρουσίασε στην οθόνη
```



Άσκηση 14

Ελληνικές γραμματοσειρές και σύμβολα στην οθόνη OLED.

Για υποστήριξη ελληνικών θα χρησιμοποιήσουμε μια άλλη βιβλιοθήκη η οποία βρίσκεται στην διεύθυνση <https://github.com/easytarget/microPyEZfonts> και συγκεκριμένα το αρχείο **ezFBfont.py** το οποίο αντιγράφεται στο σύστημα αρχείων του esp32.

Για μετατροπή των χαρακτήρων θα χρησιμοποιήσουμε το πρόγραμμα **bdf2dict.py** και για αρχείο εισόδου πρέπει να χρησιμοποιήσουμε αρχείο τύπου bdf. Το πρόγραμμα εκτελείται στο υπολογιστή. Από τον κατάλογο Unicode/unifont_15.1.05 κατεβάζουμε το αρχείο unifont_15.1.05.bdf στον ίδιο φάκελο με το πρόγραμμα bdf2dict.py και φτιάχνουμε ένα αρχείο με τους χαρακτήρες που θέλουμε στην γραμματοσειρά μας και το ονομάζουμε charset.txt π.χ. `!"#$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMNPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~ΑΒΓΔΕΖΗΘΙΚΛΜΝΞΟΠΡΣΤΥΦΧΨΩαβγδεζηθικλμνξοπρστυφχψωΆΈΉΊΌΥΰάέήίόύώς`, τέλος κάνουμε την μετατροπή γράφοντας:

```
python bdf2dict.py unifont-15.1.05.bdf my_charset.txt
```

Το αρχείο το οποίο θα παραχθεί το αντιγράφουμε στο filesystem του esp32. Αν δεν πρόκειται να χρησιμοποιήσουμε κάποιους χαρακτήρες τους αφαιρούμε από το charset.txt ώστε να καταναλώνει λιγότερη μνήμη.

```
1 from machine import Pin, I2C
2 from ssd1306 import SSD1306_I2C
3 from ezFBfont import ezFBfont
4
5 import my_unifont_15_1_05 as font1 #Εισαγωγή μεταφρασμένης γραμματοσειράς
6
7 i2c = I2C(0, sda = Pin(21), scl = Pin(22), freq = 400000)
8 display = SSD1306_I2C( width=128, height=64, i2c=i2c, addr=0x3c, external_vcc=False )
9 display.fill(0) #Καθαρίζει οθόνη
10
11 #Δημιουργία στιγμιότυπου
12 font = ezFBfont(display, font1,
13                 halign='center',
14                 valign='center',
15                 hgap=1,
16                 verbose=True)
17
18 font.write("Καλημέρα\nΚόσμε \nAbcdefg12345\n+*/#|^&.", 63, 31) #Εγγραφή στο Framebuffer
19 display.show() #Παρουσίασε στην οθόνη
```

Μπορούμε να μετατρέψουμε ttf ή otf fonts από τον υπολογιστή μας με το εργαλείο **otf2bdf** το οποίο υπάρχει στο linux ή με online ιστότοπους μετατροπής. Για μια γραμματοσειρά ttf γράφουμε:

```
otf2bdf -p 8 DejaVuSerif-Bold.ttf -o DejaVuSerif-Bold.bdf
```

Το **-p 8** είναι οι στιγμές δηλαδή το μέγεθος της γραμματοσειράς. Αν δεν το ορίσουμε τότε το καθορισμένο είναι **12pts**.

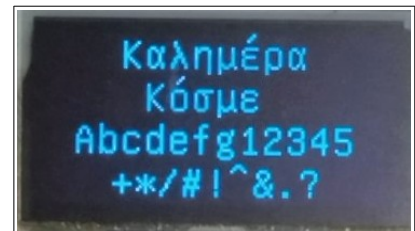
Μετά το μετατρέπουμε σε py με την ίδια εντολή όπως πριν:

```
python bdf2dict.py -p 8 DejaVuSerif-Bold.bdf my_charset.txt
```

Με το πρόγραμμα **FontForge** το οποίο είναι διαθέσιμο και για Windows, διορθώνουμε ατέλειες στο αρχείο bdf και αποθηκεύουμε με **‘Δημιουργία Γραμματοσειρών’**.

Δημιουργώντας διαφορετικά στιγμιότυπα της κλάσης ezFBfont μπορούμε να έχουμε πολλές γραμματοσειρές στην ίδια οθόνη.

Δοκιμάστε επίσης γραμματοσειρές από τον κατάλογο Unicode/efont-unicode-bdf-0.4.2 και τα σύμβολα όπως μπαταρίες, cursors, βελάκια, ψηφία 7 τμημάτων κλπ. από τον κατάλογο Symbols στο δέντρο του github.



Άσκηση 15

Εικόνες στην οθόνη OLED.

Για να εμφανίσουμε εικόνες στην οθόνη OLED αρχικά ζωγραφίζουμε μια εικόνα σε ένα πρόγραμμα ζωγραφικής π.χ. το Gimp. Οι διαστάσεις είναι 128x64 και φροντίζουμε να είναι ασπρόμαυρο (2 τόνοι). Το αποθηκεύουμε με επέκταση png ή jpg. Μπορούμε να βρούμε και μια έτοιμη εικόνα και να αλλάξουμε τις διαστάσεις σε 128x64.

Από το github πάμε στην διεύθυνση <https://github.com/TimHanewich/MicroPython-SSD1306> και κατεβάζουμε το αρχείο **src/convert.py**. Επίσης εγκαθιστούμε την βιβλιοθήκη **pillow** με **pip install pillow**. Βάζουμε τις εικόνες στον ίδιο φάκελο με το convert.py. Μέσα από τον φάκελο ανοίγουμε ένα shell της python3. Εκεί γράφουμε:

```
>>> import convert
>>> converted = convert.image_to_buffer(r"hello.png")
>>> buffer = converted[0]
>>> buffer
```

Εδώ βλέπουμε ένα μεγάλο byte array το οποίο ξεκινάει με `b'\x00\x00\x00\...` και τελειώνει με `!`. Αυτό το αντιγράφουμε ώστε να το μεταφέρουμε στο πρόγραμμά μας.

Στην 2η εντολή δίπλα στο όνομα του αρχείου μπορούμε να βάλουμε και 2η παράμετρο. Σ' αυτή ορίζουμε το κατώφλι με το οποίο αποφασίζει αν το pixel είναι άσπρο ή μαύρο.

[illegible]

Στο πρόγραμμα αυτό έχουμε δύο εικόνες οι οποίες εναλλάσσονται με περίοδο 1sec. Τα bytearray hello_buf και hello_buf1 μπορούμε να τα βάλουμε σε ένα άλλο αρχείο .py και να τα κάνουμε import στην αρχή του προγράμματος.